

A. Model Architecture and Hyperparameters

We train all models on ESM2-8M, where $L = 6$ and $d_{\text{model}} = 320$. Additionally, we train our models on 5 million random sequences under 1022 residues from UniRef50. Before mapping each layer’s residual stream activations into their respective latent spaces, we first apply the TopK function. For the CLT and PLT, we set $k = 16$ at each layer to achieve a total sparsity of $16 \times 6 = 96$ latents, which is consistent with the level of sparsity targeted by (Ameisen et al., 2025). We additionally set $d_{\text{latent}} = 3200$, which is a $10\times$ expansion factor from the hidden dimension of ESM2-8M. Basing our hyperparameters off of (Gao et al., 2025), we additionally set $k_{\text{aux}} = \lfloor d_{\text{ESM}}/2 \rfloor$ and $\alpha = 1/32$. We note that the auxk loss’s primary objective is to mitigate dead latents by forcing them to activate. For computational efficiency, we choose to activate them using the decoder matrix $\mathbf{W}_{\text{dec}}^{\ell \rightarrow \ell}$.

We train all our models with a batch size of 16 and a learning rate of 2×10^{-4} with the Adam optimizer over 250,000 steps. To help stabilize training, we utilize a gradient clipping value of 1 and weight decay of value 1×10^{-5} . We additionally normalize the mean-squared error at each layer by dividing by the variance of the ground-truth residual stream activations at each batch.

We note that a popular method for constraining the sparsity of CLT activations is using a tanh loss (Ameisen et al., 2025). While an interesting future direction, this adds significant computational burden, as it requires extensive hyperparameter tuning to balance the regularizer’s strength across different architectures. Furthermore, a tanh loss would result in varying sparsity levels between models, which would complicate experiments and hinder direct comparisons between the CLT and PLT. Adopting TopK in this paper allows us to bypass these complexities and set up a controlled environment for model evaluation.

B. Per-layer Transcoders

To benchmark our CLT, we train a per-layer transcoder (PLT), where each layer has a single transcoder trained independently of each other layer (Ameisen et al., 2025; Dunefsky et al., 2024). Here, each layer’s residual stream activations $\mathbf{x}^\ell$ are mapped to their respective $\mathbf{a}^\ell$ via:

$$\mathbf{a}^\ell = \text{TopK}(\mathbf{W}_{\text{enc}}^\ell(\mathbf{x}^\ell - \mathbf{b}_{\text{pre}}^\ell) + \mathbf{b}_{\text{enc}}^\ell). \quad (4)$$

Mathematically, the PLT encoder mapping is identical to the CLT encoder mapping from Equation (1). However, computing $\hat{\mathbf{y}}$ now uses a single decoder which only has access to the information found in layer ℓ :

$$\hat{\mathbf{y}}^\ell = \mathbf{W}_{\text{dec}}^{\ell \rightarrow \ell} \mathbf{a}^\ell + \mathbf{b}_{\text{pre}}^\ell. \quad (5)$$

For consistency in notation, we denote $\mathbf{W}_{\text{dec}}^{\ell \rightarrow \ell}$ as the PLT’s decoder at layer ℓ , but note that for each layer, there exists only a single decoder matrix unlike in a CLT.

The total training objective, $\mathcal{L}_{\text{PLT}}$, is similar to the CLT training objective and is defined as $\mathcal{L}_{\text{PLT}} = \mathcal{L}_{\text{MSE}} + \alpha \mathcal{L}_{\text{aux}}$. Since the PLT does not have access to cross-layer decoders, our setup is identical to training L -many independent transcoders.

C. Replacement Models

In this section, we describe how we design replacement models to approximate $\mathbf{y}^L$ using both CLTs and PLTs. First, we denote transformer notation to describe the forward pass of ESM2. Then, we go over the ProtoMech and PLT replacement models.

C.1. Transformer Notation

Following a similar notation established in (Elhage et al., 2021; Dunefsky et al., 2024), we represent the forward pass of a transformer as follows. Let $\mathbf{x}_{\text{pre}}^\ell$ denote the residual stream activations at the beginning of layer ℓ before applying attention. First, the attention sublayer computes its output based on $\mathbf{x}_{\text{pre}}^\ell$ and adds it to the residual stream to produce $\mathbf{x}^\ell$:

$$\mathbf{x}^\ell = \mathbf{x}_{\text{pre}}^\ell + \sum_{h \in H_\ell} h(\mathbf{x}_{\text{pre}}^\ell), \quad (6)$$

where H_ℓ represents the set of attention heads at layer ℓ and $h(\cdot)$ denotes the function computed by head h . Next, the MLP layer $\text{MLP}^\ell(\cdot)$ takes in $\mathbf{x}^\ell$ to produce $\mathbf{y}^\ell$, which is the target our CLTs are trained to reconstruct:

$$\mathbf{y}^\ell = \text{MLP}^\ell(\mathbf{x}^\ell). \quad (7)$$

Finally, $\mathbf{y}^\ell$ is added back to the residual stream to produce $\mathbf{x}_{\text{pre}}^{\ell+1}$:

$$\mathbf{x}_{\text{pre}}^{\ell+1} = \mathbf{x}^\ell + \mathbf{y}^\ell. \quad (8)$$

C.2. ProtoMech Replacement Models

In the ProtoMech, we test the following replacement models for circuit discovery: direct, sequential, and full replacement. As reconstruction errors can compound when replicating the forward pass of ESM2 (Ameisen et al., 2025), these replacement models differ in how they combat the compounding reconstruction error. In the main text, we compare the performance of ProtoMech and PLT using the *sequential* replacement model on circuit discovery tasks to ensure we can compare the representational power of both architectures fairly. However, for the steering results in the main text, we utilize the *direct* replacement model in ProtoMech (keeping the sequential replacement model for the PLT), as our primary concern is designing the highest-functioning variants. For a full comparison of results across all replacement models, we refer to Appendix E.

Direct. The direct model approximates $\mathbf{y}^L$ directly without going through the forward pass. Here, we assume we have access to all ground-truth residual stream activations $\mathbf{x}^\ell$ at each layer. Based on Equation (2), to compute $\hat{\mathbf{y}}^L$, we then need to compute the summation over all decoders from that map from $\ell' = 1, 2, \dots, L$. This method is only valid for CLTs (and not PLTs) because ablating latents in the CLT will change the output of the last layer even with ground-truth residual stream activations, while it will not change with the PLT.

Sequential. The sequential model does not assume access to all ground-truth residual stream activations $\mathbf{x}^\ell$. Here, we assume we have access to only $\mathbf{x}_{\text{pre}}^0$ as well as the ground-truth attention computations at each layer $h \in H_\ell$. Given this, the sequential model aims to replicate the entire forward pass through ESM. Given $\mathbf{x}_{\text{pre}}^0$, $\mathbf{x}^0$ is computed via Equation (6). The replacement model then approximates $\mathbf{y}^0$ as $\hat{\mathbf{y}}^0$ and, following Equation (8), produces the approximation of the next residual stream as $\hat{\mathbf{x}}_{\text{pre}}^1 = \mathbf{x}^0 + \hat{\mathbf{y}}^0$. From here, to obtain an approximation of $\hat{\mathbf{x}}^1$, we assume ground-truth access to $h(\mathbf{x}_{\text{pre}}^1)$, and proceed the same as before. We repeat the process until reaching $\hat{\mathbf{y}}^L$.

Full Replacement. Using terminology from (Merullo et al., 2025), we define the *full replacement model* as the model that replaces $\mathbf{y}^\ell$ with its CLT reconstruction at each layer ℓ . The full replacement model is similar to the sequential model, except it does not assume access to anything except $\mathbf{x}^0$. Given $\mathbf{x}_{\text{pre}}^0$, we compute $\hat{\mathbf{x}}_{\text{pre}}^1$ as in the sequential model. However, to obtain $\hat{\mathbf{x}}^1$, we do not assume access to $h(\mathbf{x}_{\text{pre}}^1)$ and instead utilize $h(\hat{\mathbf{x}}_{\text{pre}}^1)$. We repeat the process until reaching $\hat{\mathbf{y}}^L$.

Note that for both the sequential and full replacement model, error propagates throughout the forward pass, as the activations are found via:

$$\hat{\mathbf{a}}^\ell = \text{TopK}(\mathbf{W}_{\text{enc}}^\ell(\hat{\mathbf{x}}^\ell - \mathbf{b}_{\text{pre}}^\ell) + \mathbf{b}_{\text{enc}}^\ell),$$

which occurs at every layer. Conversely, the direct model bypasses this and reconstructs $\mathbf{a}^\ell$ by passing the ground truth value of $\mathbf{x}^\ell$ at each layer through the encoder.

C.3. PLT Replacement Models

In the PLT, we test the sequential and full replacement models for circuit discovery.

Sequential. The sequential model is the analog to the ProtoMech sequential model. The only values we have access to are $\mathbf{x}_{\text{pre}}^0$ and the ground-truth attention outputs for each $h \in H_\ell$. Similar to the ProtoMech sequential model, the residual stream’s attention update is done using the ground-truth attention values, while the MLP approximation $\hat{\mathbf{y}}^\ell$ is done using the PLT trained for layer ℓ .

Full Replacement. Similarly to the ProtoMech full replacement model, the MLP of ESM replaced with the PLT trained at a given layer, and no information is given other than $\mathbf{x}_0$.

We note that no analog to the direct replacement model exists for the PLT. This is due to the lack of cross-layer connections: ablation at a layer $\ell' < \ell$ does not affect the reconstruction of $\hat{\mathbf{y}}^\ell$. Accordingly, the best way to measure the effect of ablating latents on the output of a PLT is to use a replacement model that compounds errors between sequential layers.

D. Experimental Setup

Discovery with Supervised Models. Throughout this paper, we use supervised models to anchor circuit discovery to biologically relevant tasks. While circuit discovery in natural language is typically anchored to next-token prediction (Ameisen et al., 2025), we argue that supervised tasks provide a more principled objective for ESM2. For one, ESM2 is a masked language model, meaning that it is not designed for next-token prediction like in autoregressive models. Additionally, pLMs like ESM2 are primarily utilized to encode global biological properties such as protein family or function. Consequently, anchoring circuits to next-amino acid predictions would fail to capture the structural and functional motifs that drive these attributes. We believe that the supervised tasks discussed in this paper are a representative subset of the areas circuit discovery would be utilized in pLMs.

D.1. Protein Family Classification Circuit Discovery

Data. To identify family circuits, we use Swiss-Prot (Boeckmann et al., 2003) protein sequences obtained from (Adams et al., 2025). Here, Swiss-Prot protein sequences under 1022 residues were clustered at at 30% sequence identity using MMseqs2 (Steinegger & Söding, 2017). We collected and mean-pooled the MLP output from layer L across all sequences in the dataset.

Logistic Regression Probe Training. We identified protein families by their InterPro ID (Paysan-Lafosse et al., 2022). For each protein family with 50 or more protein sequences, which we set to ensure we have enough data to discover a robust circuit, we assemble training data for the probe as the protein family sequences plus randomly sampled $4\times$ the number of sequences not from that protein family. We performed a stratified split to partition the data into training, validation, and test sets. We use 90% of the data for our training set. From the remaining 10% of data, we selected up to 128 sequences to form a validation set, which is reserved specifically for circuit discovery and computing attribution scores. We then trained a logistic regression probe for a maximum of 1000 iterations with the *balanced* class weight toggle using `scikit-learn` and computed all F1 scores based on the test set.

Circuit Discovery. Following a similar procedure to (Nainani et al., 2025), we quantitatively define a circuit as the smallest subset of latents required to achieve a comparable F1 score to the original logistic regression probe, m_{clean} . For each protein family, we define the comparable F1 score, θ , to be 70% of the original F1 score. However, there may be cases where even adding all the latents do not recover 70% of the original F1 score, due to the nature of lossy reconstruction. To combat this, we compute the F1 score using CLT and PLT reconstructions with all latents active, m_{all} , and adjust the target to be the maximum possible performance achievable: $\theta = \min(0.7 \times m_{\text{clean}}, m_{\text{all}})$.

To identify the subset of latents, we employ an iterative greedy search based on gradient attribution. First, we compute an attribution score for every latent i at every layer ℓ , denoted as Δ_i^ℓ , with respect to the probe’s prediction target y . We compute the attribution score as a product of the latent’s activation a_i^ℓ and its gradient with respect to the prediction of y :

$$\Delta_i^\ell = \left| a_i^\ell \cdot \frac{\partial y}{\partial a_i^\ell} \right|. \quad (9)$$

Δ_i^ℓ estimates the contribution of each latent to the model’s output. We sum these attribution scores across all family sequences in the held-out validation set to obtain a global attribution ranking for each latent.

From here, we rank all latents by descending attribution score. We iteratively construct candidate circuits by adding the top-ranked latents in steps of 32 at a time, up to a maximum of 1000 latents. At each step, we compute the circuit performance m_{circuit} using the sparse reconstruction derived from the top latents, and continue until $m_{\text{circuit}} \geq \theta$ or until we reach 1000 latents. We note that for computational efficiency, we opted to keep the validation set relatively small. However, increasing the number of family sequences in the validation set will likely increase the quality of the circuit discovered.

Plotting. To create Fig. 1c and Fig. 5a, we only include the results of protein families with over 50 sequences, as to ensure the statistics we report are robust. These circuits are used to plot the distribution of latents in Fig. 6a. To create Fig. 8, we pull from all families regardless of size, but restrict the protein families where the F1 score is between 0.01 and 0.5. We specify that the F1 score should be above 0.01, as an F1 score of 0 means that ESM2 is not learning any information about the protein family. This means that circuit discovery will not distill any meaningful computations. We leave all protein family circuits on our HuggingFace.